EEVDF Linux Process Scheduler implementation via sched_ext

Shago Pavel Evgenevich

Saint Petersburg State University

CPS 2026

April 9

Introduction

CPU scheduling is a core mechanism of OS process management
OS scheduler governs how to allocate processor time among competing tasks

Linux Kernel

- Open-source operating system kernel
- Manages hardware resources
- Widely used across computing domains

The OS Scheduler

- Subsystem for task selection and CPU allocation
- Determines execution order and task's time slicing
- Governs resource utilization, responsiveness, and predictability

Problem

- The built-in Linux EEVDF scheduler is part of the kernel and cannot be modified without recompilation and a full system reboot
- This work investigates reimplementation of EEVDF via sched_ext to study how different implementation approach can affect scheduling performance
- To evaluate the implementation, standard workload benchmarks are used such as hackbench, sysbench, and eBPF-based scheduling latency tracing
- Then the sched_ext variant is compared against the built-in kernel EEVDF to identify trade-offs introduced by eBPF dispatch

sched_ext (scheduler extensions) is a new Linux kernel infrastructure that enabled implementing CPU scheduling policies as eBPF programs attached to kernel scheduling hooks at runtime

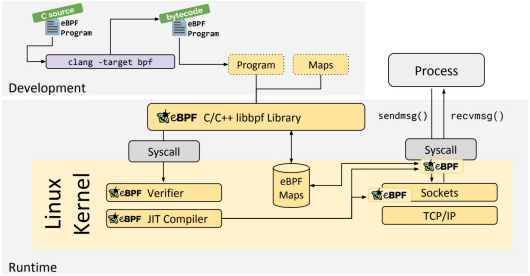

Figure 1. eBPF in the network subsystem

DSQs (dispatch queues):

- SCX_DSQ_LOCAL – per-core fast path
- SCX_DSQ_GLOBAL – shared queue

Scheduler cycle:

- select_cpu – pick idle cpu core
- enqueue – place task in global DSQ
- dispatch – move to local DSQ
- running / stopping – based on vtime

Earliest Eligible Virtual Deadline First Model

The ideal reference system is such system where every active client i continuously receives share $w_i/W(t)$ of the CPU. EEVDF tracks this entitlement through system virtual time $V(t)$,

$$W(t) = \sum_{j \in A(t)} w_j, \qquad V(t) = \int_{t_0}^{t} \frac{d\tau}{W(\tau)}.$$

When client i submits a request of size r, the algorithm assigns a *virtual eligible time* and a *virtual deadline*,

$$v_e = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i}, \qquad v_d = v_e + \frac{r}{w_i},$$

where $s_i(t_0^i, t)$ is the service received since t_0^i.

Earliest Eligible Virtual Deadline First Model

A request is runnable only once $v_e \leq V(t)$. Among all eligible requests, EEVDF dispatches the one with the smallest virtual deadline v_d.

A client behind its proportional share carries a smaller v_e and becomes eligible sooner. Larger weights compress r/w_i, so heavier tasks earn earlier deadlines for the same request size.

The lag of client i measures the signed deviation between ideal model entitlement and actual service received. EEVDF guarantees [1] $|\text{lag}_i(t)| < q$ in steady state, the optimal bound for any discrete proportional-share scheduler with quantum q.

Mapping to sched_ext

Client	$\rightarrow$	task_struct
w_i	$\rightarrow$	p->scx.weight
v_i	$\rightarrow$	scx.dsq_vtime
$V(t)$	$\rightarrow$	vtime_now
r	$\rightarrow$	SCX_SLICE_DFL

S - integer scale

enqueue:

$$VE_i = \max(v_i,\ V_{\text{now}} - \Delta)$$

$$VD_i = VE_i + \frac{r \cdot S}{w_i} \rightarrow \text{insert by } VD_i$$

running:

$$V_{\text{now}} \leftarrow \max(V_{\text{now}},\ v_i)$$

stopping:

$$v_i \leftarrow v_i + \frac{\text{consumed} \cdot S}{w_i}$$

Environment

System

Kernel:
Linux 7.0-rc4

CPU:
Intel Core
Ultra 7 265K

Comparison

Built-in EEVDF
vs
sched_ext EEVDF

Benchmarks

- **hackbench** – 1000 tasks
  Groups of processes exchange messages intensively, simulating high system load. Reflects context-switching and task coordination efficiency.
- **sysbench** – default parameters
  CPU, memory, and disk I/O throughput. Simulates OLTP database operations under realistic conditions.
- **sched-latency** – custom eBPF
  Attaches to tracepoints and records p99 schedule latency.

Metrics

hackbench
Total time (s)

sysbench
$Events/s$

sched-latency
p99 lat (μs)

Results

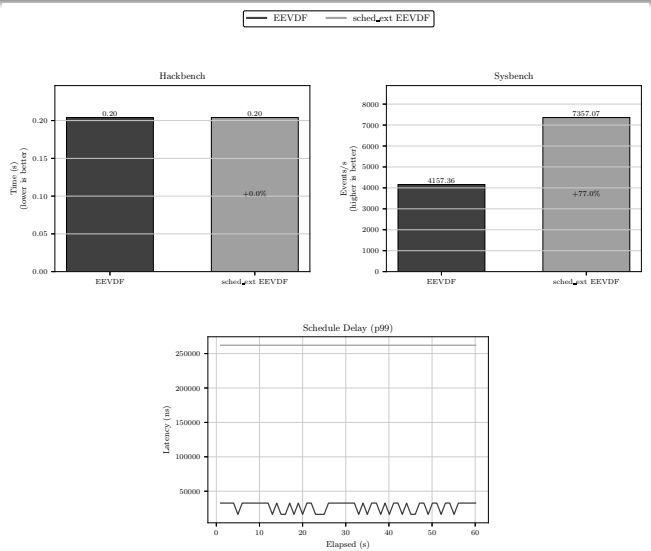

Figure 2. Test results for builtin EEVDF scheduler and implemented sched_ext EEVDF

Analysis

- **Hackbench:** 0.20 s for both schedulers (+0.0%)

→ eBPF dispatch overhead is negligible for intensive, scheduler demanding workloads

- **Sysbench:** 4157 → 7357 events/s

→ +77% – may offer advantages for sustained computational workloads

- **Scheduling latency (p99):** 25 μs → 250 μs

→ 10× increase attributable to eBPF dispatch overhead and contention on the single shared DSQ

Proposed sched_ext EEVDF achieves in-kernel EEVDF performance parity while decoupling scheduling policy from the kernel and providing greater flexibility

Conclusion

- EEVDF scheduling algorithm is faithfully reproducible outside the kernel via sched_ext and eBPF
- Policy changes take effect without recompilation or reboot, enabling rapid iteration
- eBPF dispatch overhead is significant, overall performance parity with in-kernel EEVDF is achievable
- Results lay the groundwork for future research targeting heterogeneous CPU platforms

References & Q/A

[1] Stoica I., Abdel-Wahab H. Earliest eligible virtual deadline first. PhD thesis. Old Dominion University, 1995

[2] An EEVDF CPU scheduler for Linux (LWN)

[3] The extensible scheduler class (LWN)